

Relative path: classification/classifier.py

Filename: classifier.py Extension: .py

"""

OASIS v2 – Layer 2: Anomaly-Type Classifier

=====

Why a separate model from detection (ARCHITECTURE.md Section 3)

Detection (Layer 1) answers "is this session anomalous?" as an unsupervised problem specifically so it never has to deal with class imbalance. This layer answers "which of the 7 attack types does it resemble?" – a genuinely supervised, multi-class question – but it ONLY EVER RUNS on sessions Layer 1 already flagged. That pre-filtering is what keeps this layer imbalance-safe: it never sees the overwhelming "normal" majority at all, so there's no 99:1 class skew for a RandomForest to be fooled by.

Features

Reuses the same numeric feature vector as the baseline profiler (profiling.baseline_model.FEATURE_NAMES) plus a few classification-specific signals that the profiler doesn't compute (privileged-action presence, distinct source IPs, distinct entity_types seen from the same source_ip in the current batch) – anything that's informative for *which* attack type, not just *whether* it's anomalous.

Honest limitation

This hackathon's synthetic dataset only injects ~4-8 examples per attack type. A RandomForest trained/evaluated on that few examples per class will have noisy per-class precision/recall – real numbers, not fabricated ones, but reported with that caveat attached (see eval/metrics.py and the report's "known limitations" section).

"""

from __future__ import annotations

from typing import Dict, List

import numpy as np

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from profiling.baseline_model import FEATURE_NAMES, extract_features

```
EXTRA_FEATURE_NAMES = [
    "has_privileged_action",
    "num_distinct_ips",
    "resource_breadth",
    "geo_velocity_flag",
    "rarity_score",
    "drift_score",
    "device_recurrence_count",
]
```

ALL_FEATURE_NAMES = FEATURE_NAMES + EXTRA_FEATURE_NAMES

```
PRIVILEGED_ACTIONS = {"grant_admin", "access_admin_panel", "modify_permissions",
                      "access_root_shell", "disable_logging"}
```

CONFIDENCE_THRESHOLD = 0.4

```
def extract_classification_features(
    session: dict,
    known_resources: set,
    known_geos: set = frozenset(),
    detection_signals: dict | None = None,
    device_recurrence: dict | None = None,
) -> np.ndarray:
    base = extract_features(session, known_resources, known_geos)
    signals = detection_signals or {}
    recurrence_key = (session["entity_id"], session["device_fingerprint"])
    recurrence_count = (device_recurrence or {}).get(recurrence_key, 1)
    extra = np.array([
        1.0 if any(a in PRIVILEGED_ACTIONS for a in session["command_sequence"]) else 0.0,
        len(set(session["source_ips"])),
        len(set(session["resources_touched"])),
    ])
```

```

        1.0 if signals.get("geo_velocity_flag") else 0.0,
        float(signals.get("rarity_score", 0.0)),
        float(signals.get("drift_score", 0.0)),
        float(recurrence_count),
    ])
    return np.concatenate([base, extra])

class AnomalyClassifier:
    def __init__(self, seed: int = 42):
        self.seed = seed
        self.clf = RandomForestClassifier(
            n_estimators=200, max_depth=6, min_samples_leaf=1,
            class_weight="balanced", random_state=seed,
        )
        self.classes_: List[str] = []
        self._fitted = False

    def fit(
        self,
        labeled_sessions: List[dict],
        labels: List[str],
        known_resources_by_entity: dict,
        known_geos_by_entity: dict | None = None,
        detection_signals_by_session: dict | None = None,
        device_recurrence: dict | None = None,
    ):
        """`labeled_sessions` must be ATTACK-labeled sessions only (never
        'normal' / 'insider_drift') – see module docstring."""
        known_geos_by_entity = known_geos_by_entity or {}
        detection_signals_by_session = detection_signals_by_session or {}
        X = np.array([
            extract_classification_features(
                s,
                known_resources_by_entity.get(s["entity_id"], set()),
                known_geos_by_entity.get(s["entity_id"], set()),
                detection_signals_by_session.get(s["session_id"]),
                device_recurrence,
            )
            for s in labeled_sessions
        ])
        y = np.array(labels)
        self.classes_ = sorted(set(labels))
        if len(labeled_sessions) >= 10 and min(np.unique(y, return_counts=True)[1]) >= 2:
            X_train, X_test, y_train, y_test = train_test_split(
                X, y, test_size=0.3, random_state=self.seed, stratify=y)
        else:
            X_train, y_train = X, y
            X_test, y_test = X, y # too few samples per class to hold out honestly

        self.clf.fit(X_train, y_train)
        self._fitted = True
        return X_test, y_test

    def predict(
        self,
        session: dict,
        known_resources: set,
        known_geos: set = frozenset(),
        detection_signals: dict | None = None,
        device_recurrence: dict | None = None,
    ):
        x = extract_classification_features(
            session,
            known_resources,
            known_geos,
            detection_signals,
            device_recurrence,
        ).reshape(1, -1)
        probs = self.clf.predict_proba(x)[0]
        classes = self.clf.classes_
        best_idx = int(np.argmax(probs))
        best_prob = float(probs[best_idx])
        class_probabilities = {c: round(float(p), 4) for c, p in zip(classes, probs)}
        if best_prob < CONFIDENCE_THRESHOLD:
            return "uncertain", best_prob, class_probabilities

```

```
return classes[best_idx], best_prob, class_probabilities
```